Virtual Audio Router

VST3 虚拟音频路由套件 — 开发文档

版本 2.1.0 — 内部参考

1. 项目结构

```
D:\wdm2vst\
├── ASMRTOP_Driver_V2/          # Windows 内核驱动（WDM/WaveRT）
│   ├── Source/
│   │   ├── Main/               # 驱动入口、miniport、内核 IPC
│   │   ├── Inc/                # 头文件、branding.h
│   │   ├── Filters/            # 音频过滤器实现
│   │   └── Utilities/          # 工具函数
│   ├── VirtualAudioDriver.sln  # VS2022 解决方案
│   ├── build.bat               # 编译脚本
│   └── set_brand.ps1           # 品牌切换脚本
│
├── ASMRTOP_Plugins/            # VST3 插件（基于 JUCE C++）
│   ├── Source/
│   │   ├── W2V_Plugin*.cpp/h   # WDM2VST 插件
│   │   ├── V2W_Plugin*.cpp/h   # VST2WDM 插件
│   │   ├── Send_Plugin*.cpp/h  # Audio Send 插件
│   │   ├── Recv_Plugin*.cpp/h  # Audio Receive 插件
│   │   ├── AsmrtopIPC.h        # 共享内存 IPC 桥接
│   │   ├── P2PTransport.h      # UDP 发送/接收 + WebSocket 服务器
│   │   ├── Localization.h      # 中英文本地化
│   │   ├── PluginBranding.h    # 品牌宏定义
│   │   ├── QRCode.h            # 二维码生成器（无外部依赖）
│   │   └── QRCodePopup.h       # 二维码弹窗组件
│   ├── JUCE/                   # JUCE 框架（子模块）
│   └── CMakeLists.txt          # CMake 构建配置
```

2. 系统架构

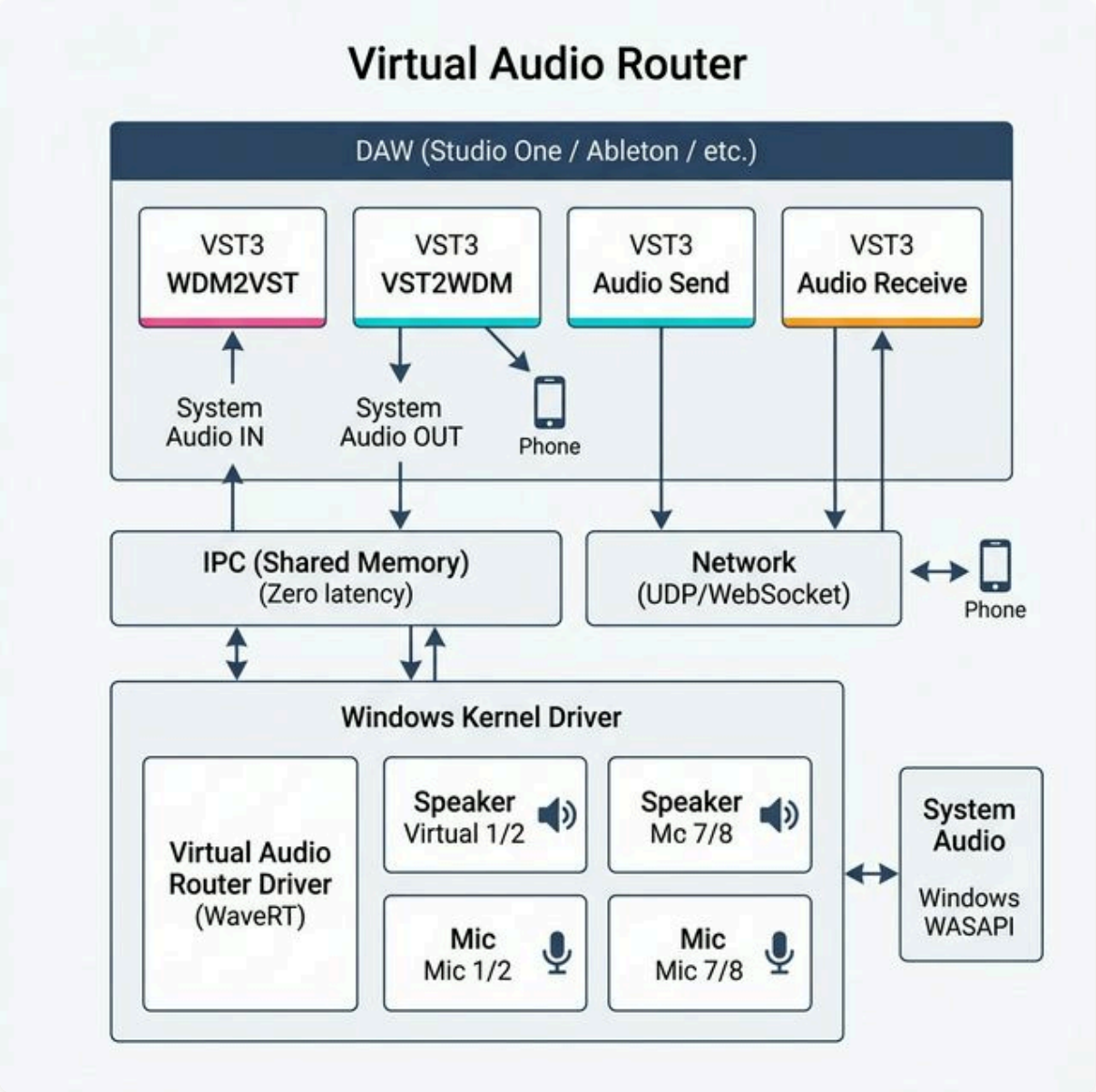

图 1：系统架构

2.1 IPC 数据流（零延迟）

```
DAW processBlock() → SharedMemoryBridge::write() → 内核驱动读取
内核驱动写入 → SharedMemoryBridge::read() → DAW processBlock()
```

- 使用 Windows 命名共享内存：`Global\VirtualAudioWDM_PLAY_0`
- 无锁环形缓冲区，原子读写指针
- 数据包格式：32 字节头 + 交错 float32 PCM 数据

2.2 网络数据流（UDP）

```
Send::processBlock() → P2PSender → DatagramSocket → UDP 数据包
UDP 数据包 → DatagramSocket → P2PReceiver → Recv::processBlock()
```

- 使用 16 位 PCM 编码以节省带宽
- 默认端口：18810
- 支持广播（.255）和单播

2.3 手机数据流（WebSocket）

```
processBlock() → pushToWebSocketClients() → TCP WebSocket 帧
手机浏览器 ← WebSocket ← 内置 HTTP/WS 服务器
```

- 手写 HTTP 服务器 + WebSocket 握手（无外部依赖）
- 内联 SHA-1 计算 Sec-WebSocket-Accept
- 二进制帧：16 位 PCM 音频数据
- 浏览器端使用 JavaScript AudioWorklet 播放

3. 核心组件

3.1 共享内存桥接（AsmrtopIPC.h）

```
struct SharedAudioPacket {
    uint32_t magic;       // 0x41534D52 ("ASMR")
    uint32_t version;     // 协议版本
    uint32_t channelId;   // 通道 ID (0-3)
    uint32_t sampleRate;  // 采样率
    uint32_t numSamples;  // 样本数
    uint32_t numChannels; // 固定为 2（立体声）
    uint32_t writePos;    // 写指针（原子）
    uint32_t readPos;     // 读指针（原子）
    float data[RING_BUFFER_SAMPLES * 2]; // 交错 L/R
};

class SharedMemoryBridge {
    // baseName: "VirtualAudioWDM"（公版）或 "AsmrtopWDM"（品牌版）
    // direction: "PLAY"（扬声器方向）/ "REC"（麦克风方向）/ "P2P"
    SharedMemoryBridge(baseName, direction, channelId);
    void write(const float* left, const float* right, int numSamples);
    int read(float* left, float* right, int maxSamples);
    int getAvailableSamples();
};
```

命名规则： `Global\<baseName>_<direction>_<channelId>`

3.2 P2P 传输层（P2PTransport.h，约 660 行）

P2PSender 管理三种传输：

1. **IPC** — SharedMemoryBridge（P2P 方向）
2. **UDP** — DatagramSocket 发送到局域网
3. **WebSocket** — 内置 HTTP + WS 服务器（手机监听）

P2PReceiver 管理两种传输：IPC 读取 + UDP 监听

3.3 本地化系统（Localization.h）

```
#define ZH(s) juce::String::fromUTF8(s)

class Lang {
    enum ID { CHANNEL, MODE, STATUS, START, STOP, ... };
    static juce::String get(ID id);  // 根据当前语言返回字符串
    static void toggle();            // 中英文切换
    static juce::String t(ID id);    // get() 的简写
};
```

- 使用 `juce::String::fromUTF8()` 配合原始 UTF-8 十六进制序列
- 标题字符串根据 `PluginBranding.h` 宏自动适配品牌

3.4 品牌系统

通过 `set_brand.ps1` 脚本切换 `PluginBranding.h` 中的 `#define`：

宏	公版	品牌版
PLUGIN_COMPANY_NAME	"VirtualAudioRouter"	"ASMRTOP"
PLUGIN_IPC_BASE_NAME	"VirtualAudioWDM"	"AsmrtopWDM"
VST3 文件名	WDM2VST.vst3 等	ASMRTOP WDM2VST.vst3 等

> **重要**：插件的 IPC 名称必须与驱动的 IPC 名称一致。两个品牌的插件和驱动不可混用！

4. 编译指南

4.1 前置条件

工具	版本要求
Visual Studio	2022（v143 工具集）
CMake	3.20 以上
Windows SDK	10.0.22621.0 以上
WDK	仅编译驱动时需要

4.2 编译插件

```
# 切换品牌
powershell -EP Bypass -File set_brand.ps1 public

# 配置
cd ASMRTOP_Plugins
cmake -B build -S .

# 编译 Release 版
cmake --build build --config Release
```

输出路径：`build/<TARGET>_artefacts/Release/VST3/<插件名>.vst3/`

4.3 编译两个版本

```
# 1. 编译品牌版
set_brand.ps1 asmrtop → cmake 编译 → 收集到 output/ASMRTOP/
```

```
# 2. 编译公版
set_brand.ps1 public → cmake 编译 → 收集到 output/Public/
```

4.4 编译驱动

```
cd ASMRTOP_Driver_V2
build.bat
```

需要 WDK 和代码签名证书（正式发布用）。

5. 插件实现细节

5.1 WDM2VST — processBlock 流程

1. 检查是否 IPC 模式 → 从 SharedMemoryBridge 读取
2. 否则从 WASAPI 环形缓冲区读取（由 audioDeviceIOCallback 填充）
3. 应用增益 + 限制器
4. 更新峰值电平表

5.2 VST2WDM — processBlock 流程

1. 应用增益 + 限制器
2. 写入 WASAPI 环形缓冲区（由 audioDeviceIOCallback 消费）
3. 写入 IPC SharedMemoryBridge（如果是 IPC 模式）
4. 推送到 WebSocket 客户端（如果手机监听已启用）
5. 更新峰值电平表

5.3 Audio Send

管理 `P2PSender` 生命周期。processBlock 将音频推送到所有已启用的传输层。

5.4 Audio Receive

管理 `P2PReceiver` 生命周期。processBlock 从 IPC/UDP 环形缓冲区读取音频。

6. 关键设计决策

1. **头文件库** — IPC、P2P、QR、本地化均为 header-only，便于 CMake 集成
2. **无外部网络依赖** — WebSocket/HTTP 服务器完全手写，仅依赖 JUCE Socket
3. **UTF-8 十六进制转义** — 避免源文件编码在不同编辑器/编译器间产生问题
4. **编译期品牌切换** — 单一代码库，通过宏生成两个产品
5. **无锁环形缓冲区 IPC** — 内核与用户态之间零延迟音频传输
6. **16 位 PCM 网络传输** — 带宽仅为 float32 的一半，监听场景质量足够

7. 文件参考

文件	行数	用途
AsmrtopIPC.h	~120	共享内存环形缓冲区桥接
P2PTransport.h	~660	UDP 收发 + WebSocket 服务器
Localization.h	~155	中英文字符串表
PluginBranding.h	~30	品牌宏定义
QRCode.h	~300	二维码生成器（无依赖）
QRCodePopup.h	~60	二维码弹窗 JUCE 组件
set_brand.ps1	~160	品牌切换 PowerShell 脚本